

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO BÀI TẬP LỚN
**PHÁT TRIỂN HỆ THỐNG ĐẤU
GIÁ TRỰC TUYẾN THỜI GIAN
THỰC (REAL-TIME BIDDING)**

Môn học: Lập trình nâng cao (UET.CS2043)
Mã học phần: CS2043 - Nhóm lớp: K70I-IT5

Họ và tên	Mã sinh viên	Vai trò chính
Nguyễn Văn Hiếu	25020156	Phát triển Client (JavaFX) & Common
Đỗ Hải Đăng	25020112	Phát triển Backend System & Common
Nguyễn Hữu Hanh	25020141	Tester, Xử lý ngoại lệ & Làm báo cáo
Lê Đình Minh Anh	25020012	Database, Thiết kế DAO & Quay video

Hà Nội, Tháng 06 năm 2026

Mục lục

1	Giới thiệu mục tiêu và phạm vi thực hiện	2
1.1	Mục tiêu và Bài toán thực tế	2
1.2	Phạm vi thực hiện và Công nghệ	2
2	Kiến trúc tổng thể hệ thống (Architecture)	2
2.1	Mô hình Multi-Module và Separation of Concerns	2
2.2	Luồng xử lý yêu cầu (Request Flow)	2
3	Xử lý Concurrency chuyên sâu	3
3.1	Race Condition và Pessimistic Locking	3
3.2	Deadlock Prevention (Chống khóa chết)	3
4	Logic nghiệp vụ và Thuật toán nâng cao	3
4.1	Thuật toán Auto-Bidding (Đấu giá tự động)	3
4.2	Anti-sniping (Chống bắn tã)	4
5	Thiết kế Cơ sở dữ liệu và Toàn vẹn dữ liệu	4
5.1	Schema Design (ERD)	4
5.2	Data Integrity at DB Tier	4
6	Phân chia công việc và Quy trình phát triển	4
6.1	Phân chia vai trò (Team Contributions)	4
6.2	Kết luận và Đánh giá	5

1 Giới thiệu mục tiêu và phạm vi thực hiện

1.1 Mục tiêu và Bài toán thực tế

Đấu giá trực tuyến là một trong những ứng dụng khắc nghiệt nhất đối với các nhà phát triển phần mềm do tính chất "nhảy cảm" về thời gian và giá trị tài chính. Một hệ thống đấu giá chuyên nghiệp phải giải quyết được 3 thách thức lớn:

- Độ trễ (Latency):** Thông tin giá phải được đồng bộ hóa tức thời tới hàng nghìn thiết bị.
- Tính nhất quán (Consistency):** Tuyệt đối không để xảy ra tình trạng ghi đè giá (Race Condition) hoặc số dư tài khoản bị âm.
- Tính công bằng (Fairness):** Ngăn chặn các hành vi trục lợi như bắn tỉa giây cuối (Sniping) hoặc thao túng giá.

1.2 Phạm vi thực hiện và Công nghệ

Hệ thống được phát triển bám sát mô hình Desktop App hiện đại:

- **Ngôn ngữ:** Java 21 (LTS) - Tận dụng các tính năng mới nhất về hiệu năng.
- **Frontend:** JavaFX 21 + FXML + CSS - Đảm bảo giao diện mượt mà, responsive.
- **Backend:** Java Core Socket (Multi-threading), NIO Selector.
- **Persistence:** MySQL 9.0 + HikariCP Connection Pool.
- **Security:** BCrypt (hashing), Pessimistic Locking (CSDL).

2 Kiến trúc tổng thể hệ thống (Architecture)

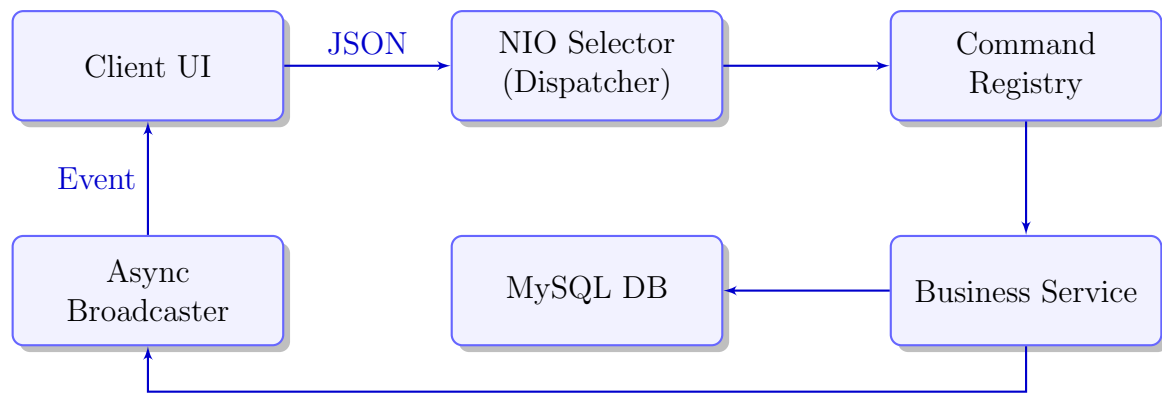
2.1 Mô hình Multi-Module và Separation of Concerns

Hệ thống được module hóa triệt để nhằm tối ưu khả năng bảo trì:

- **Module common:** Chứa toàn bộ các Class thực thể (Entity), hằng số và DTO. Việc sử dụng chung module này đảm bảo tính thống nhất trong serialization/deserialization JSON.
- **Module server:** Một máy chủ TCP hướng sự kiện, chịu trách nhiệm thực thi toàn bộ logic nghiệp vụ và quản lý trạng thái phiên.
- **Module client:** Ứng dụng GUI vận hành độc lập, giao tiếp với server qua kết nối Socket duy nhất được duy trì (Persistent Connection).

2.2 Luồng xử lý yêu cầu (Request Flow)

Hệ thống áp dụng **Command Pattern** kết hợp với **Reactor Pattern** ở tầng mạng:



Hình 1: Luồng xử lý Command-Based thời gian thực

3 Xử lý Concurrency chuyên sâu

3.1 Race Condition và Pessimistic Locking

Trong hệ thống đấu giá, Race Condition xảy ra khi 2 người dùng cùng đặt giá cho một phiên tại cùng một thời điểm micro-giây. Nếu xử lý thông thường, giá trị cuối cùng có thể bị sai lệch. **Giải pháp:** Sử dụng **Pessimistic Locking** ở tầng CSDL.

```

1 txManager.execute(conn -> {
2     Auction auction = auctionDao.getAuctionForUpdate(conn, auctionId);
3     if (amount <= auction.getCurrentPrice()) throw new ValidationException("Gia thap!");
4
5     User bidder = userDao.findByAccountnameForUpdate(conn, bidderName);
6     return result;
7 });
  
```

3.2 Deadlock Prevention (Chống khóa chết)

Deadlock xảy ra khi Luồng A giữ khóa User 1 và đợi User 2, trong khi Luồng B giữ User 2 và đợi User 1. **Giải pháp: Lock Ordering Rule.** Hệ thống luôn sắp xếp danh sách tài khoản cần khóa theo thứ tự bảng chữ cái trước khi thực hiện truy vấn.

```

1 List<String> accountsToLock = Arrays.asList(bidderName, sellerName, prevWinner);
2 Collections.sort(accountsToLock);
3 for (String acc : accountsToLock) {
4     userDao.findByAccountnameForUpdate(conn, acc);
5 }
  
```

4 Logic nghiệp vụ và Thuật toán nâng cao

4.1 Thuật toán Auto-Bidding (Đấu giá tự động)

Hệ thống hỗ trợ cơ chế ủy thác đặt giá. Thuật toán hoạt động theo nguyên tắc *Vickrey-style modification*:

1. Server nhận lệnh đặt giá thủ công của User A.
2. Trong cùng một Transaction, Server duyệt danh sách các lệnh AutoBid đã cài đặt trước đó.

3. Nếu có AutoBid (User B) với `maxBid` cao hơn mức giá hiện tại, Server tự động thực hiện lệnh đặt giá cho User B với bước giá tối thiểu cần thiết để dẫn đầu.
4. Quá trình này lặp lại cho đến khi không còn AutoBid nào có thể vượt qua mức giá hiện tại hoặc ngân sách của họ đã cạn.

4.2 Anti-sniping (Chống bắn tỉa)

Bắn tỉa là hành vi đợi đến giây cuối cùng để đặt giá khiến người khác không kịp phản ứng. Hệ thống giải quyết bằng **Dynamic Time Extension**:

- **Window**: 30 giây cuối cùng của phiên.
- **Action**: Nếu có bất kỳ lượt đặt giá hợp lệ nào trong Window này, `endTime` của phiên tự động được cộng thêm 60 giây.
- **Implementation**: Sử dụng `AuctionMonitor` tích hợp `ScheduledExecutorService` để tự động hủy task kết thúc cũ và tái lập lịch task mới một cách chính xác.

5 Thiết kế Cơ sở dữ liệu và Toàn vẹn dữ liệu

5.1 Schema Design (ERD)

Cơ sở dữ liệu được thiết kế chuẩn hóa mức 3 (3NF) để tránh dư thừa dữ liệu:

- **Table users**: Lưu trữ `balance` (tổng tiền) và `blocked_balance` (tiền đang dùng để giữ chỗ đấu giá).
- **Table auctions**: Lưu `current_price`, `winner_accountname` và `version` (cho Optimistic Locking).
- **Table transactions**: Lưu mọi biến động dòng tiền phục vụ đối soát.

5.2 Data Integrity at DB Tier

Sử dụng các ràng buộc (Constraints) mạnh để bảo vệ dữ liệu khỏi lỗi logic ứng dụng:

```
1 ALTER TABLE users ADD CONSTRAINT chk_funds CHECK (balance >= 0);
2 ALTER TABLE auctions ADD CONSTRAINT chk_time CHECK (end_time > start_time);
3 ALTER TABLE bids ADD CONSTRAINT chk_bid_val CHECK (bid_amount > 0);
```

6 Phân chia công việc và Quy trình phát triển

6.1 Phân chia vai trò (Team Contributions)

Dự án được triển khai dựa trên sự phân cấp trách nhiệm rõ ràng, đảm bảo tính hỗ trợ và phối hợp hiệu quả giữa các cấu phần:

- **Nguyễn Văn Hiếu:** Chịu trách nhiệm chính trong việc phát triển ứng dụng Client sử dụng JavaFX; xây dựng hệ thống giao diện trực quan và cơ chế non-blocking UI giúp giao tiếp mạng thời gian thực diễn ra mượt mà; đồng thời phối hợp thiết kế module `common` để định nghĩa cấu trúc dữ liệu dùng chung cho hệ thống.
- **Đỗ Hải Đăng:** Chịu trách nhiệm thiết kế và hiện thực lõi máy chủ Backend System; tối ưu cấu trúc đa luồng, kiểm soát tương tranh và an toàn giao dịch tài chính tăng nghiệp vụ; đồng thời phối hợp tổ chức module `common` nhằm chuẩn hóa giao thức payload JSON giữa hai đầu hệ thống.
- **Nguyễn Hữu Hanh:** Đảm nhiệm vai trò kỹ sư đảm bảo chất lượng (Tester); xây dựng kịch bản unit test toàn diện cho các dịch vụ cốt lõi, kiểm soát hệ thống bọ lỗi và xử lý ngoại lệ đồng bộ; đồng thời chịu trách nhiệm tổng hợp tiến độ, biên tập dữ liệu kỹ thuật để hoàn thiện báo cáo đồ án.
- **Lê Đình Minh Anh:** Phụ trách thiết kế lược đồ cơ sở dữ liệu MySQL bảo đảm tính toàn vẹn dữ liệu; hiện thực các mẫu thiết kế DAO (Data Access Object) phục vụ việc đọc ghi và khóa bản ghi tối ưu; đồng thời chịu trách nhiệm sản xuất và biên tập video thực tế mô phỏng toàn bộ kịch bản luồng hoạt động của hệ thống.

6.2 Kết luận và Đánh giá

Hệ thống đã đạt được mục tiêu đề ra với khả năng xử lý tương tranh chuyên nghiệp, đảm bảo an toàn tài chính cho người dùng. Các tính năng nâng cao như Anti-sniping và Auto-bidding đã hoạt động ổn định trong môi trường mô phỏng đa luồng. Đây là minh chứng cho việc áp dụng đúng đắn các kiến thức về Lập trình nâng cao, Cấu trúc dữ liệu và Hệ điều hành vào thực tiễn.